



UNIVERSIDAD AUTONOMA DE NAYARIT



Unidad Académica de Economía

Carrera

Licenciatura en Sistemas Computacionales

Grupo y Semestre

Semestre 6

Nombre estudiante

Ricardo Matos Vizcarra

Actividad

S2 Reto IA 1

Maestro

Eligardo Cruz Sánchez

Materia

Programación Distribuida del lado Cliente

UNIVERSIDAD AUTÓNOMA DE NAYARIT

Diseño de Enpoints:

Método HTTP	Endpoint (URL)	Acción/Propósito
GET	/api/productos	Obtiene la lista de todos los productos (soporta filtros y paginación).
POST	/api/productos	Crea un nuevo producto en el catálogo.
GET	/api/productos/{id}	Obtiene los detalles de un producto específico mediante su ID.
PUT	/api/productos/{id}	Actualiza de forma completa la información de un producto.
PATCH	/api/productos/{id}	Realiza una actualización parcial (ej. solo cambiar el precio).
DELETE	/api/productos/{id}	Elimina un producto del sistema.



Recurso: Productos

Operación	Método	Endpoint	Body de ejemplo	Respuesta Exitosa	Posibles errores
Crear	POST	/productos	{"nombre": "Miel Orgánica", "precio": 12.50, "productor_id": 5}	201 Created	400 Bad Request
Listar	GET	/productos	N/A	200 OK (Lista de objetos)	500 Server Error
Buscar	GET	/productos?nombre=Miel	N/A	200 OK (Filtrados)	400 Bad Request
Obtener	GET	/productos/{id}	N/A	200 OK (Objeto único)	404 Not Found
Actualizar	PUT	/productos/{id}	{"nombre": "Miel Flores", "precio": 13.0, "productor_id": 5}	200 OK	400, 404
Parcial	PATCH	/productos/{id}	{"precio": 14.0}	200 OK	400, 404
Eliminar	DELETE	/productos/{id}	N/A	204 No Content	404 Not Found

Recurso: Productores

Operación	Método	Endpoint	Body de ejemplo	Respuesta Exitosa	Posibles errores
Crear	POST	/productores	{"nombre": "Granja El Sol", "region": "Andes"}	201 Created	400 Bad Request
Listar	GET	/productores	N/A	200 OK	500 Server Error
Anidado	GET	/productores/{id}/productos	N/A	200 OK (Lista de productos)	404 Not Found
Eliminar	DELETE	/productores/{id}	N/A	204 No Content	409 Conflict (Si tiene productos)

Recurso: Pedidos

Operación	Método	Endpoint	Body de ejemplo	Respuesta exitosa	Posibles errores
Crear	POST	/pedidos	{"cliente_id": 1, "items": [{"producto_id": 2, "cantidad": 3}]}	201 Created	400, 422 (Sin stock)
Listar	GET	/pedidos	N/A	200 OK	500 Server Error
Obtener	GET	/pedidos/{id}	N/A	200 OK	404 Not Found
Estado	PATCH	/pedidos/{id}	{"estado": "enviado"}	200 OK	400 Bad Request
Cancelar	DELETE	/pedidos/{id}	N/A	204 No Content	404, 403 (No permitido)

Comparativa de las tablas

Al comparar ambos diseños, el que tiene mayor propuesta técnica y de utilidad para el desarrollo e implementación es la que presento la IA, ya que la propuesta de diseño del primero es muy básica pero general y lo que se necesita es que sea mas específica para el desarrollo, como un tipo de documento técnico de requerimientos.

Características	Mi propuesta	Propuesta IA	Ganador
Alcance del Negocio	Solo describe un recurso (Productos).	Define un ecosistema completo (Productos + Productores + Pedidos).	Propuesta IA
Contratos de Datos	No especifica qué datos enviar. Solo describe la acción.	Incluye ejemplos de Body (JSON), lo cual es vital para el programador de Frontend y Backend.	Propuesta IA
Manejo de Estados	Ignora los códigos de respuesta HTTP.	Especifica respuestas exitosas (201, 204) y errores (400, 404, 409, 422).	Propuesta IA
Lógica de Relaciones	No muestra cómo se conectan los datos.	Incluye endpoints anidados (/productores/{id}/productos), demostrando jerarquía.	Propuesta IA
Reglas de Negocio	Ninguna.	Identifica validaciones del mundo real (ej. No borrar productor si tiene productos, o error 422 si no hay stock).	Propuesta IA

¿Por qué la de la IA es más funcional para implementar?

Si tuvieras que escribir el código de la API. la propuesta IA te da las respuestas a las preguntas críticas que la primera propuesta ignora:

¿Qué campos espero en el JSON? (La IA te dice: nombre, precio, productor_id).

¿Qué pasa si intento borrar algo que está amarrado a otra tabla? (La IA define el error 409 Conflict).

¿Cómo manejo un carrito de compras? (La IA define la estructura de items con producto_id y cantidad).

Mis decisiones en diseño

1. El uso de DELETE para "Cancelar" (Recurso Pedidos)

En tu tabla de Pedidos, usas DELETE /pedidos/{id} para la operación "Cancelar".

En sistemas transaccionales (como una tienda), los registros rara vez se eliminan físicamente por temas de auditoría y contabilidad.

Usar un PATCH /pedidos/{id} con un body {"estado": "cancelado"}. Si realmente quieres un endpoint dedicado a la acción, podrías usar POST /pedidos/{id}/cancelar, pero el PATCH es más fiel al estándar REST para cambios de estado. El DELETE lo reservaría solo si el pedido es un borrador que nunca se procesó.

2. Consistencia en la Respuesta de Actualización (Recurso Productos)

En Productos, para PUT y PATCH, se indica una respuesta 200 OK.

Aunque es correcto, en el diseño de APIs modernas se prefiere que las operaciones de actualización devuelvan el objeto completo actualizado en el cuerpo de la respuesta, no solo el código de estado.

Quiero asegurarnos de que en el OpenAPI el esquema de respuesta para 200 incluya el modelo del producto. Esto evita que el frontend tenga que hacer un GET adicional para refrescar la interfaz tras un cambio de precio, por ejemplo.